Trabajo Práctico Integrador

Desarrollo de Sistemas Web (Front-End)

Carrera: Tecnicatura Superior en Desarrollo de Software
Materia: Desarrollo de Sistemas Web (Front-End)
Instancia: Trabajo Práctico Integrador / Parcial
Modalidad: Individual
Presentación presencial: Miércoles 24 de junio de 2026, en el IFTS
Entrega obligatoria en Moodle: Martes 23 de junio de 2026

---

1. Presentación general

El presente Trabajo Práctico Integrador constituye la instancia de evaluación parcial de la materia **Desarrollo de Sistemas Web (Front-End)**.

El objetivo del trabajo es que cada estudiante desarrolle una página web completa de complejidad introductoria, integrando los contenidos trabajados durante la cursada:

- HTML
- CSS
- JavaScript
- Diseño responsive
- Interacción con el usuario
- Validación básica de formularios
- Organización del código

La propuesta busca evaluar no solamente el resultado visual del sitio, sino también la comprensión de la estructura, el uso correcto de las tecnologías y la capacidad de explicar el trabajo realizado.

---

2. Modalidad de trabajo

El trabajo será de carácter **individual**.

Cada estudiante deberá desarrollar un sitio web o landing page funcional sobre un tema a elección, dentro de las opciones propuestas o previa aprobación del docente.

El trabajo deberá poder abrirse correctamente en un navegador web a partir del archivo index.html.

No se aceptarán trabajos que dependan exclusivamente de plataformas externas, constructores visuales o herramientas que oculten el código trabajado.

---

3. Fechas importantes

Entrega en Moodle

El trabajo deberá subirse obligatoriamente al campus Moodle hasta el día:

Martes 23 de junio de 2026

La entrega en Moodle es condición obligatoria para aprobar la instancia.

El/la estudiante que no suba el trabajo al Moodle en la fecha indicada no aprobará la instancia de parcial, aunque asista a la presentación presencial.

No se aceptarán trabajos enviados únicamente por correo electrónico, WhatsApp, enlaces sueltos, pendrive u otros medios no indicados por el docente.

Presentación presencial

La presentación presencial se realizará el día:

Miércoles 24 de junio de 2026, en el IFTS

Ese día cada estudiante deberá mostrar su trabajo funcionando y explicar las decisiones tomadas durante el desarrollo.

Esta presentación corresponde a la instancia de **parcial** de la materia.

---

4. Consigna general

Desarrollar una página web completa utilizando **HTML, CSS y JavaScript**, orientada a presentar un servicio, emprendimiento, institución, profesional, producto o actividad.

El sitio deberá tener:

- estructura clara;
- contenido organizado;
- diseño visual coherente;
- adaptación a distintos tamaños de pantalla;
- interacción básica mediante JavaScript;
- formulario con validación;
- código prolijo y entendible.

El trabajo deberá representar una producción completa, no una suma de ejemplos aislados.

5. Temas sugeridos

El/la estudiante podrá elegir uno de los siguientes temas:

- Sitio web de una clínica médica.
- Sitio web de una clínica veterinaria.
- Landing page de una barbería o peluquería.
- Sitio de un gimnasio o centro deportivo.
- Página de presentación de un curso.
- Sitio de un emprendimiento gastronómico.
- Landing page de un profesional independiente.
- Sitio de una tienda o catálogo simple.
- Página institucional de una organización.
- Sitio de una consultora o agencia.
- Sitio de una actividad cultural o evento.
- Otro tema similar, previa aprobación del docente.

El tema elegido debe permitir desarrollar una página con contenido suficiente para aplicar HTML, CSS y JavaScript.

---

6. Requisitos mínimos de HTML

El sitio deberá incluir una estructura HTML correcta y ordenada.

Como mínimo, deberá contener:

- estructura básica de documento HTML;
- uso correcto de DOCTYPE;
- etiqueta html;
- sección head;
- título de la página;
- enlace al archivo CSS;
- enlace al archivo JavaScript;
- encabezado principal;
- menú de navegación;
- contenido principal;
- secciones diferenciadas;
- formulario;
- pie de página.

Se espera el uso de etiquetas semánticas, por ejemplo:

```
<header>
<nav>
```

```
<main>
<section>
<article>
<footer>
<form>
<label>
<input>
<textarea>
<button>
```

No es obligatorio usar todas esas etiquetas, pero sí debe verse una intención clara de organizar correctamente el contenido.

---

7. Contenido mínimo del sitio

La página deberá incluir, como mínimo:

1. Encabezado

Debe contener el nombre del sitio, marca, emprendimiento, institución o servicio.

También puede incluir un logo, imagen o texto identificatorio.

2. Menú de navegación

Debe permitir acceder visualmente a las principales secciones del sitio.

Ejemplo:

- Inicio
- Servicios
- Nosotros
- Galería
- Contacto

No hace falta que sean páginas separadas. Puede ser una página única con navegación interna.

3. Sección de presentación

Debe explicar brevemente de qué trata el sitio.

Puede incluir:

- título principal;
- subtítulo;
- breve descripción;
- imagen;
- botón de acción.

4. Sección de servicios, productos o información principal

Debe mostrar al menos tres elementos relevantes.

Ejemplos:

- servicios de una clínica;
- cortes de una barbería;
- cursos disponibles;
- productos destacados;
- actividades de un gimnasio;
- prestaciones de un profesional.

5. Tabla, listado o bloque organizado de información

Debe incluir información estructurada.

Ejemplos:

- horarios de atención;
- precios;
- planes;
- turnos disponibles;
- cronograma;
- comparación de servicios;
- datos de contacto;
- catálogo simple.

6. Formulario

Debe incluir un formulario de contacto, inscripción, reserva o consulta.

Como mínimo deberá tener:

- nombre;
- correo electrónico o teléfono;
- campo de consulta o mensaje;
- botón de envío.

Puede agregar otros campos según el tema elegido.

7. Pie de página

Debe incluir información final del sitio.

Ejemplos:

- datos de contacto;
- redes sociales;
- dirección;
- derechos reservados;
- enlaces internos.

---

8. Requisitos mínimos de CSS

El sitio deberá contar con una hoja de estilos propia llamada:

`style.css`

El diseño deberá incluir, como mínimo:

- colores definidos de manera coherente;
- tipografías legibles;
- márgenes;
- padding;
- bordes;
- espaciado entre secciones;
- estilos para títulos y textos;
- estilos para botones;
- estilos para tarjetas, bloques o secciones;
- estilos para el formulario;
- diseño responsive;
- uso de Flexbox y/o Grid.

El sitio deberá verse correctamente tanto en pantalla de escritorio como en dispositivos móviles.

No se evaluará solamente que el sitio “se vea lindo”, sino que el diseño esté construido de manera ordenada y que el/la estudiante pueda explicar cómo aplicó los estilos.

---

9. Diseño responsive

El trabajo deberá adaptarse a diferentes tamaños de pantalla.

Como mínimo, el sitio deberá verse correctamente en:

- computadora de escritorio;
- tablet o pantalla mediana;
- celular.

Se valorará el uso de:

- Flexbox;
- Grid;
- porcentajes;
- unidades relativas;
- media queries;
- imágenes adaptables;
- menús o bloques que se acomoden en pantallas pequeñas.

Ejemplo de media query:

```
@media (max-width: 768px) {
  .contenedor {
    flex-direction: column;
  }
}
```

El diseño responsive no significa solamente que “entre en la pantalla”, sino que el contenido siga siendo legible, ordenado y usable.

---

10. Requisitos mínimos de JavaScript

El sitio deberá incluir un archivo JavaScript propio llamado:

`script.js`

El trabajo deberá incorporar interacción básica con el usuario.

Como mínimo, deberá incluir:

- selección de elementos del DOM;
- uso de eventos;
- al menos una función propia;
- validación básica de formulario;
- modificación de algún contenido, mensaje, clase o estilo mediante JavaScript.

Ejemplos válidos:

- validar que los campos obligatorios estén completos;
- mostrar un mensaje de confirmación al enviar el formulario;
- mostrar un error si falta completar un dato;
- mostrar u ocultar información;
- cambiar contenido dinámicamente;
- activar una sección desplegable;
- abrir o cerrar un modal;
- cambiar una imagen o texto según una opción;
- calcular un dato simple ingresado por el usuario;
- aplicar una clase CSS desde JavaScript.

No alcanza con incluir un archivo script.js vacío o con un único alert() sin relación real con el sitio.

---

11. Uso de Bootstrap, frameworks o librerías CSS

Se permite el uso de **Bootstrap**, Tailwind u otros frameworks o librerías CSS como herramienta de apoyo.

Sin embargo, el uso de un framework no reemplaza el trabajo propio del estudiante.

El sitio deberá incluir obligatoriamente una hoja de estilos propia llamada:

`style.css`

Allí el/la estudiante deberá personalizar el diseño.

¿Qué significa usar Bootstrap con estilos propios?

Bootstrap puede utilizarse para facilitar:

- grillas;
- columnas;
- botones base;
- formularios;
- tarjetas;
- menú responsive;
- espaciados;
- componentes visuales.

Pero el/la estudiante deberá agregar estilos propios para personalizar el sitio.

Por ejemplo, se puede usar una clase de Bootstrap:

```
<button class="btn btn-primary">
  Solicitar turno
```

```
</button>
```

Pero eso solo genera un botón estándar de Bootstrap.

Para que haya trabajo propio, se espera agregar una clase personalizada:

```
<button class="btn btn-primary boton-turno">
  Solicitar turno
</button>
```

Y luego modificarla en style.css:

```
.boton-turno {
  background-color: #7c3aed;
  border: none;
  border-radius: 30px;
  padding: 12px 24px;
  font-weight: bold;
}

.boton-turno:hover {
  background-color: #5b21b6;
}
```

En ese caso, Bootstrap aporta una base, pero el diseño final fue personalizado por el/la estudiante.

Ejemplo con tarjeta

Uso básico de Bootstrap:

```
<div class="card">
  <div class="card-body">
    <h3 class="card-title">Consulta médica</h3>
```

```
    <p class="card-text">Atención profesional con turno previo.</p>
  </div>
</div>
```

Uso de Bootstrap con estilos propios:

```
<div class="card tarjeta-servicio">
  <div class="card-body">
    <h3 class="card-title">Consulta médica</h3>
    <p class="card-text">Atención profesional con turno previo.</p>
  </div>
</div>
```

CSS propio:

```
.tarjeta-servicio {
  border-radius: 20px;
  border: 1px solid #e5e7eb;
  background-color: #f9fafb;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.tarjeta-servicio:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}
```

Este tipo de personalización sí será considerado válido.

Qué está permitido con Bootstrap o frameworks

Está permitido:

- usar container, row, col, card, btn, navbar, form-control u otras clases;
- usar la grilla responsive;
- usar componentes como cards, formularios, botones o navbar;
- adaptar componentes de Bootstrap al tema elegido;
- combinar clases de Bootstrap con clases propias;
- modificar colores, tamaños, bordes, espaciados y efectos;
- crear estilos propios en style.css;
- usar Bootstrap como apoyo, no como reemplazo del trabajo.

Qué no será suficiente

No será suficiente:

- entregar una página hecha solamente con clases de Bootstrap;
- usar componentes sin ninguna personalización;
- descargar una plantilla y cambiar únicamente los textos;
- copiar una landing page completa sin comprenderla;
- entregar código que el/la estudiante no pueda explicar;
- usar Bootstrap para evitar aplicar CSS propio.

Una página construida exclusivamente con Bootstrap, sin estilos propios y sin comprensión del código, no cumplirá con los requisitos del trabajo.

---

12. Uso de inteligencia artificial

Se permite el uso de herramientas de inteligencia artificial como apoyo durante el desarrollo del trabajo.

El uso de IA no está prohibido, pero deberá ser responsable, transparente y acompañado por comprensión real del código entregado.

Usos permitidos de inteligencia artificial

Se puede utilizar IA para:

- consultar dudas;
- pedir explicaciones;
- revisar errores;
- detectar problemas en el código;
- pedir ideas de diseño;
- mejorar textos;
- generar ejemplos de referencia;
- entender cómo validar un formulario;
- consultar cómo aplicar media queries;

- pedir ayuda para ordenar el código;
- comprender errores de JavaScript.

Usos no válidos de inteligencia artificial

No será válido:

- entregar un trabajo generado completamente por IA sin comprenderlo;
- copiar y pegar código sin saber explicarlo;
- presentar como propio un sitio que no se entiende;
- usar IA para reemplazar el proceso de aprendizaje;
- depender de la IA durante la presentación presencial para responder preguntas;
- entregar código que no coincida con lo trabajado durante la cursada.

El problema no es usar herramientas. El problema es entregar algo que no se comprende.

Declaración de uso de IA

En caso de utilizar inteligencia artificial, el/la estudiante deberá incluir dentro de la entrega un archivo llamado:

`uso_ia.txt`

En ese archivo deberá indicar:

- qué herramienta utilizó;
- para qué la utilizó;
- qué partes del trabajo fueron asistidas;
- qué modificaciones realizó personalmente;
- qué aprendió o corrigió a partir de esa asistencia.

Ejemplo:

```
Herramienta utilizada: ChatGPT


Uso realizado:
Utilicé la herramienta para consultar cómo validar un formulario con JavaScript y
para revisar errores en el diseño responsive.


Partes asistidas:
- Validación del formulario de contacto.
- Corrección de errores en media queries.
- Mejora de textos del sitio.
```

```
Modificaciones propias:

Adapté el código a mi estructura HTML, cambié nombres de clases, modifiqué estilos y
ajusté los mensajes mostrados al usuario.


Comprensión:

Puedo explicar cómo se seleccionan los campos del formulario, cómo se valida si están
vacíos y cómo se muestra el mensaje final al usuario.

Si el/la estudiante no utilizó IA, puede aclararlo en el mismo archivo:

No utilicé herramientas de inteligencia artificial para el desarrollo de este
trabajo.
```

---

13. Uso de plantillas

Se recomienda desarrollar el sitio desde cero o a partir de una estructura simple creada por el/la estudiante.

El uso de plantillas completas no está prohibido de manera absoluta, pero será evaluado con especial atención.

Si se utiliza una plantilla, deberá estar profundamente modificada y el/la estudiante deberá poder explicar:

- qué plantilla usó;
- qué partes modificó;
- qué código agregó;
- qué estilos cambió;
- qué funcionalidad JavaScript implementó;
- cómo adaptó la plantilla al tema elegido.

No se aprobarán trabajos donde solamente se haya descargado una plantilla y cambiado textos, imágenes o colores mínimos.

---

14. Imágenes y recursos externos

Se permite utilizar imágenes, íconos, fuentes o recursos externos, siempre que estén correctamente integrados al proyecto.

Las imágenes deberán estar guardadas preferentemente dentro de una carpeta:

`img`

o

`assets`

Se recomienda usar imágenes livianas y con nombres claros.

Ejemplos:

```
clinica-frente.jpg
servicio-veterinaria.jpg
logo.png
```

No se recomienda usar nombres confusos como:

```
imagen1.jpg
descarga-final-ultima-version.png
fotooooo.jpg
```

También se valorará el uso correcto del atributo alt en las imágenes.

Ejemplo:

```
<img src="img/clinica.jpg" alt="Frente de la clínica veterinaria">
```

---

15. Archivos a entregar

El trabajo deberá entregarse en Moodle como archivo comprimido .zip.

El archivo .zip deberá contener, como mínimo:

- `index.html`
- `style.css`
- `script.js`
- carpeta img o assets, si se utilizan imágenes u otros recursos;
- uso_ia.txt, en caso de haber utilizado IA o para declarar que no se utilizó.

Estructura sugerida:

```
apellido_nombre_tp_integrador/
│
├── index.html
├── style.css
├── script.js
├── uso_ia.txt
│
└── img/
```

```
    └── imagenes_utilizadas
```

El archivo comprimido deberá nombrarse de la siguiente manera:

`Apellido_Nombre_TP_Integrador_FrontEnd.zip`

Ejemplo:

`Perez_Juan_TP_Integrador_FrontEnd.zip`

---

16. Condiciones técnicas de entrega

Antes de subir el trabajo, el/la estudiante deberá verificar que:

- el archivo index.html abre correctamente;
- el CSS se aplica correctamente;
- el JavaScript funciona;
- las imágenes se cargan;
- los enlaces internos funcionan;
- el formulario muestra validaciones o mensajes;
- el diseño se adapta a pantalla chica;
- la carpeta está completa;
- el archivo .zip no está vacío;
- el nombre del archivo es claro;
- el trabajo subido a Moodle es la versión final.

No se considerará como entrega válida un archivo incompleto, dañado, vacío o que no permita revisar el trabajo.

---

17. Presentación presencial

El día **miércoles 24 de junio de 2026**, cada estudiante deberá presentar su trabajo en el IFTS.

Durante la presentación, deberá mostrar el sitio funcionando y explicar el desarrollo realizado.

La presentación no consiste únicamente en mostrar la página terminada. El/la estudiante deberá demostrar que comprende el código presentado.

Durante la defensa, se podrá solicitar que explique:

- qué tema eligió;
- cómo organizó la estructura HTML;
- qué etiquetas semánticas utilizó;

- cómo aplicó CSS;
- qué estilos propios desarrolló;
- si usó Bootstrap, qué partes corresponden al framework;
- qué partes personalizó;
- cómo resolvió el diseño responsive;
- qué funcionalidad implementó con JavaScript;
- cómo funciona la validación del formulario;
- qué errores o dificultades tuvo;
- cómo los resolvió;
- si usó IA, para qué la utilizó y qué modificaciones realizó.

---

18. Criterios de evaluación

La evaluación tendrá en cuenta los siguientes aspectos:

HTML

Se evaluará:

- estructura correcta del documento;
- uso adecuado de etiquetas semánticas;
- organización clara del contenido;
- uso correcto de títulos, párrafos, listas, enlaces, imágenes, tablas o bloques de información;
- formulario correctamente armado;
- código prolijo y legible;
- nombres de clases comprensibles;
- relación coherente entre el contenido y el tema elegido.

CSS

Se evaluará:

- diseño visual coherente;
- uso correcto de selectores, clases e identificadores;
- aplicación de colores, tipografías, márgenes y espaciados;
- uso del box model;
- uso de Flexbox y/o Grid;
- adaptación responsive;
- personalización visual;
- organización del archivo style.css;
- claridad del código;
- consistencia visual entre secciones.

JavaScript

Se evaluará:

- uso correcto de variables;
- uso de funciones;
- uso de eventos;
- selección y manipulación del DOM;
- validación básica de formulario;
- modificación de contenido, clases o estilos;
- interacción clara con el usuario;
- código comprensible;
- relación entre la funcionalidad y el sitio desarrollado.

Responsive

Se evaluará:

- correcta visualización en escritorio;
- correcta visualización en celular;
- reacomodamiento de columnas, tarjetas o secciones;
- legibilidad del texto;
- usabilidad del menú y formulario;
- imágenes adaptables;
- ausencia de desbordes o elementos cortados.

Presentación y defensa

Se evaluará:

- claridad para explicar el trabajo;
- comprensión del código propio;
- capacidad para responder preguntas;
- relación entre lo entregado y lo presentado;
- explicación del uso de Bootstrap, frameworks o IA, si corresponde;
- justificación de decisiones de diseño y desarrollo.

---

19. Condiciones para aprobar

Para aprobar la instancia, el/la estudiante deberá cumplir con todas las condiciones siguientes:

1. Subir el trabajo completo al Moodle hasta el martes 23 de junio de 2026.
2. Presentarse el miércoles 24 de junio de 2026 en el IFTS.
3. Mostrar el sitio funcionando.
4. Entregar los archivos requeridos.
5. Cumplir con los requisitos mínimos de HTML, CSS y JavaScript.
6. Explicar el código y las decisiones tomadas.
7. Demostrar comprensión del trabajo realizado.

La falta de entrega en Moodle implica la no aprobación de la instancia.

La asistencia a la presentación presencial no reemplaza la entrega obligatoria en Moodle.

---

20. Situaciones que pueden afectar la aprobación

Podrán afectar negativamente la calificación o impedir la aprobación:

- no subir el trabajo a Moodle;
- entregar fuera de término;
- entregar un archivo que no abre;
- entregar un .zip incompleto;
- no incluir index.html;
- no incluir style.css;
- no incluir script.js;
- no presentar funcionalidad JavaScript real;
- no tener formulario;
- no aplicar diseño responsive;
- entregar una plantilla sin modificación suficiente;
- no poder explicar el código;
- copiar código sin comprenderlo;
- entregar un trabajo igual o prácticamente igual al de otro estudiante;
- no presentarse a la defensa presencial.

---

21. Recomendaciones de desarrollo

Se recomienda organizar el trabajo en etapas:

1. Elegir el tema.
2. Definir las secciones del sitio.
3. Crear la estructura HTML.
4. Agregar el contenido.
5. Aplicar estilos CSS generales.
6. Diseñar tarjetas, botones, menú y formulario.
7. Adaptar el sitio a celular.
8. Incorporar JavaScript.
9. Validar el formulario.

10. Probar el sitio en el navegador.
11. Revisar nombres de archivos y carpetas.
12. Crear el archivo uso_ia.txt, si corresponde.
13. Comprimir la carpeta completa.
14. Subir el .zip al Moodle antes de la fecha límite.
15. Preparar la explicación para la presentación presencial.

---

22. Ejemplo de estructura mínima esperada

Un ejemplo posible de estructura del sitio podría ser:

Inicio
- Nombre del emprendimiento o institución
- Presentación general
- Botón de contacto

Servicios
- Servicio 1
- Servicio 2
- Servicio 3

Información
- Tabla de horarios, precios o planes

Contacto
- Formulario con validación

Footer
- Datos finales

Este esquema es solo una referencia. Cada estudiante puede adaptarlo según el tema elegido.

---

23. Ejemplo de funcionalidad JavaScript válida

Una funcionalidad mínima válida podría ser:

- el usuario completa un formulario;
- JavaScript verifica que los campos no estén vacíos;
- si falta un dato, muestra un mensaje de error;
- si todo está completo, muestra un mensaje de confirmación.

Ejemplo conceptual:

```
const formulario = document.querySelector("#formulario");
```

```
const mensaje = document.querySelector("#mensaje");

formulario.addEventListener("submit", function(evento) {
  evento.preventDefault();

  const nombre = document.querySelector("#nombre").value;
  const email = document.querySelector("#email").value;

  if (nombre === "" || email === "") {
    mensaje.textContent = "Por favor, completá los campos obligatorios.";
    mensaje.classList.add("error");
  } else {
    mensaje.textContent = "Consulta enviada correctamente.";
    mensaje.classList.add("exito");
  }
});
```

No es obligatorio usar exactamente este código. Es solo un ejemplo del tipo de interacción esperada.

---

24. Importante

El trabajo debe ser una integración real de los contenidos de la materia.

No se evaluará únicamente el resultado estético. Se evaluará principalmente:

- que el sitio esté bien estructurado;
- que el CSS esté aplicado de manera coherente;
- que exista interacción real con JavaScript;
- que el diseño sea responsive;
- que el código esté organizado;
- que el/la estudiante pueda explicar lo que hizo.

El uso de herramientas como Bootstrap o inteligencia artificial está permitido, pero no reemplaza la comprensión personal del trabajo.

El/la estudiante deberá poder defender su producción y demostrar que entiende el código entregado.

---

25. Resumen de entrega

Fecha límite de entrega en Moodle:
Martes 23 de junio de 2026

Presentación presencial / Parcial:
Miércoles 24 de junio de 2026, en el IFTS

Modalidad:
Individual

Formato de entrega:
Archivo .zip

Archivos mínimos:

`index.html`
`style.css`
`script.js`
`uso_ia.txt`
carpeta img o assets, si corresponde

Condición obligatoria:
El trabajo debe estar subido al Moodle para poder aprobar la instancia.